IQRASEC ACADEMY

SQL Injection Hidden Data Report

WAPT Phase 1

Task 4 Common Web Vulnerabilities

Prepared by

Kinza Noor

Report date

3 October 2026

Learning platform

PortSwigger

Description:

This report covers the lab SQL injection vulnerability in WHERE clause allowing retrieval of hidden data. The supplied lab instance displayed Solved when checked on 10 October 2026. The original exploit request and response were not supplied, so those evidence fields remain editable.

Table of Contents

1 Objective 3
2 Concept and Understanding 3
2.1 SQL Injection 3
2.2 Query Structure 3
2.3 Payload Logic 3
2.4 Impact and Verification 3
3 Practical Work 4
3.1 Target and Scope 4
3.2 Baseline Request 4
3.3 Category Modification 4
4 Tools Used 4
5 Evidence 4
6 Result 5
7 Key Learnings 5
8 References 5

1 Objective

To understand how a product-category input can alter a database query and bypass a restriction on unreleased products. The practical objective is to compare a normal category response with the response to a modified parameter and document the hidden data returned.

2 Concept and Understanding

2.1 SQL Injection

SQL injection occurs when untrusted input affects the structure of a database statement. In this lab, the entry point is the category query parameter at /filter. The intended application behavior is to return products in the selected category while excluding unreleased records.

2.2 Query Structure

The following query is an explanatory model adapted to the Accessories category in the supplied URL. It was not recovered from the running database.

```
SELECT * FROM products
WHERE category = 'Accessories' AND released = 1
```

The category comparison selects a product group. The released condition restricts the displayed records. Concatenating an input directly into the quoted category value can allow SQL syntax to replace the intended filtering logic.

2.3 Payload Logic

```
' OR 1=1--
```

The apostrophe closes the quoted value. The OR expression adds a condition that is always true. In the lab query context, the comment marker removes the remaining restriction. The resulting query can return records beyond the permitted product set. Comment behavior is database dependent.

2.4 Impact and Verification

The lab demonstrates a confidentiality impact through disclosure of unreleased product information. It does not establish that database writes, credential extraction or server compromise occurred. A 200 response or a larger page alone is insufficient; compare the returned content and preserve the lab's success indicator.

3 Practical Work

3.1 Target and Scope

Platform: PortSwigger Web Security Academy
Endpoint: /filter
Parameter: category
Initial category: Accessories
Supplied instance host:
0ad500cc04b0b7c1817552db007f00e0.web-security-academy.net

3.2 Baseline Request

GET /filter?category=Accessories

Capture the normal request and response. If using Repeater, check HTTPS and port 443 before sending.

3.3 Category Modification

Use the documented solution below to reproduce the exercise. Record your original input separately.

Decoded parameter value: ' OR 1=1--
GET /filter?category=%27%20OR%201%3D1--

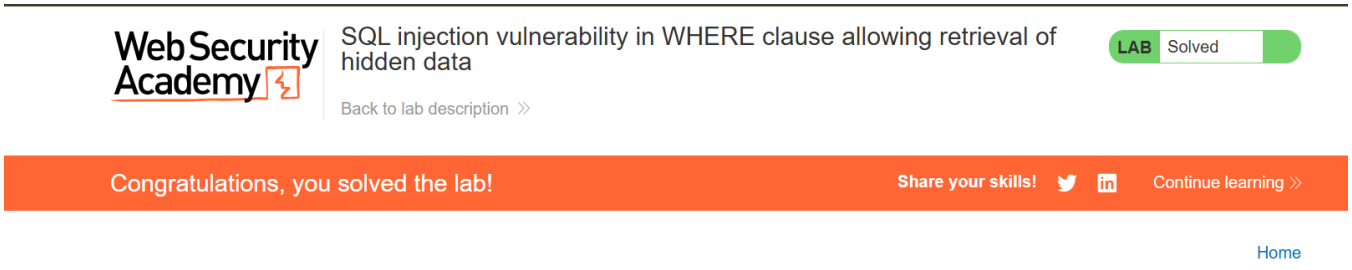
Encoding uses %27 for an apostrophe, %20 for spaces and %3D for equals. Change only category and compare responses.

4 Tools Used

Tool	Purpose
Web browser	Open the filter and inspect the product results.
Burp Proxy and Repeater if used	Capture, modify and resend the category request.
Developer Tools if used	Inspect the request and response without Burp.
PortSwigger lab instance	Provide the assigned vulnerable application.

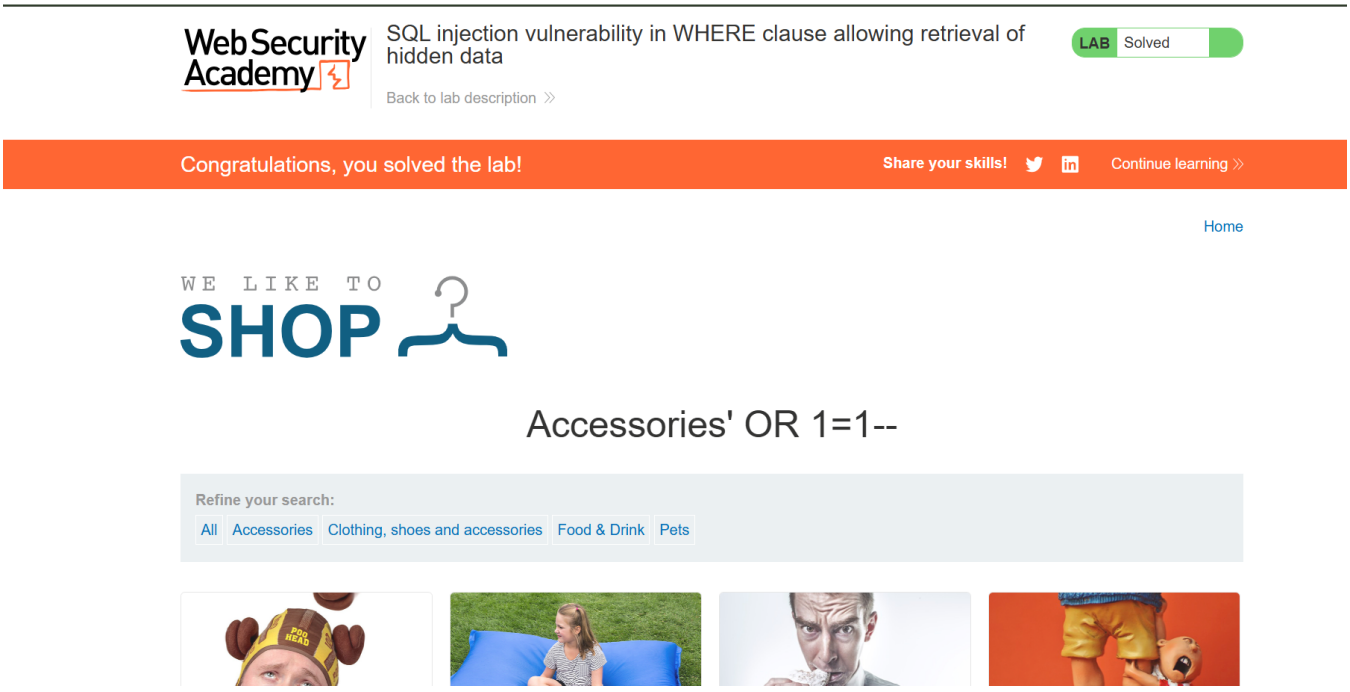
5 Evidence

Insert clear screenshots that show the request and relevant result. Keep the category input, response content and lab title readable. Redact session tokens when necessary.



6 Result

The supplied instance returned HTTP 200 for the normal category page and displayed the Solved status on 10 October 2026. This verifies the instance's completion indicator. The original exploit payload, hidden-product names and response comparison still require the learner's screenshots or saved traffic.



7 Key Learnings

Query parameters can reach database statements, so input must remain data rather than SQL syntax. A small change to a quoted value can remove a release restriction. Preserve both normal and modified responses to explain the finding, and distinguish observed completion from the exact test sequence used.

Prevent this weakness by binding category as a parameter in a fixed query and preserving the released condition. Validate categories against permitted application values as an additional control. Apply least privilege to the database account and retest that malicious-looking category text cannot disclose unreleased records. These are recommendations; no remediation was performed in this report.

8 References

PortSwigger. SQL injection lab retrieving hidden data. https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data

PortSwigger. SQL injection concepts and prevention. https://portswigger.net/web-security/sql-injection